Packing-Aware, Reference-Constrained Data Scheduling for Supervised Fine-Tuning

Anonymous Authors[1]

Abstract

Instruction-tuning mixtures contain examples with different sequence lengths, hardware costs, and effects on downstream capabilities, yet data selection and physical sequence packing are normally optimized separately. We study packing-aware, reference-constrained online scheduling: each optimizer step searches for predicted target-reference improvement while limiting regression on protected reference losses under an end-to-end compute budget. Our method uses low-dimensional, optimizer-state-conditioned gradient proxies to plan an isolated pack, then evaluates the *actual* candidate AdamW displacement on fresh gate examples before committing the update. A standard smoothness argument gives conditional one-step loss control only on a joint event of valid score, norm, and local-curvature bounds; the empirical gate does not by itself establish that event or imply general capability safety. This methods-first draft specifies the estimator, algorithm, falsification tests, and matched-budget protocol, while deliberately withholding empirical claims until the specified experiments are executed.

1. Introduction

Supervised fine-tuning (SFT) converts a pretrained language model into an instruction-following system, but the available supervision is rarely homogeneous. A single training pool may mix code, mathematics, dialogue, tool-use, and safety data; examples within each source differ by orders of magnitude in length and may induce conflicting updates. Training on every example is expensive, while selecting examples only by semantic similarity or loss can improve one target and silently regress others.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Recent selectors estimate target influence from optimizer-aware gradients (Xia et al., 2024), training trajectories (Yang et al., 2024), Shapley approximations (He et al., 2024), or Fisher information (Deb et al., 2025). Token-level cleaning further shows that useful and informative and potentially harmful supervision can coexist within one response (Pang et al., 2025). These methods largely treat the training cost of an example as additive. In practice, the cost of a batch depends on sequence length, padding, packing, kernels, and the accumulation schedule. Selecting a fixed set and packing it afterward should change execution cost but, under correct isolation, not its gradient. Under a compute budget, however, that cost changes which windows are feasible and thereby couples selection to packing through the feasible set.

A second difficulty is multi-capability control. Online weighting and mixture methods adapt data use as the model changes (Yu et al., 2025; Tan et al., 2026; Ming et al., 2026; Zhao et al., 2026), but a weighted average objective can hide a severe regression on one protected slice. Conversely, a hard constraint based only on a planning proxy may be misleading: an additive frozen-preconditioner approximation cannot reconstruct the candidate-dependent second moment of AdamW.

We ask the following falsifiable question: *after charging all selection, packing, verification, and rejected-update costs, does joint pack-level scheduling improve the target-loss versus worst-protected-loss frontier over selecting first and packing second?* The question concerns specified reference distributions, not an undefined notion of general model "safety."

This paper makes four scoped contributions:

- We formulate SFT scheduling over a complete optimizer accumulation window, with a profiled non-additive pack cost and explicit constraints for multiple protected reference losses.
- We derive an optimizer-state-conditioned planning score while making clear why per-example sketches cannot exactly predict an AdamW update.
- We apply an independent post-selection gate that evaluates the actual candidate displacement before optimizer

state is committed, and give a conditional one-step loss bound.

- We specify a matched end-to-end compute protocol designed to distinguish a genuine selection–packing interaction from selector overhead, proxy error, and ordinary best-fit packing.

Evidence status. The mathematical statements below follow from stated assumptions. All result cells remain unfilled: no throughput, loss, or benchmark advantage is claimed before running the protocol in Section 5.

2. Related Work

Offline selection and valuation. LESS stores low-rank, Adam-aware gradient features and retrieves examples aligned with target examples (Xia et al., 2024). SmallToLarge summarizes proxy model loss trajectories (Yang et al., 2024); SHED moves Shapley valuation to cluster representatives (He et al., 2024); FisherSFT selects for last-layer-linearized information gain (Deb et al., 2025); and influence distillation amortizes expensive influence computations through landmarks (Nikdan et al., 2025). Large-pool studies caution that expensive selectors can underperform simple representation-similarity selection (Ivison et al., 2025). A controlled ICML study separates representation from subset optimization and finds that gradient representations are the most consistently predictive, but that selector differences shrink as budget grows (Nayak et al., 2026). We therefore retain zero-shot and simple greedy baselines and test cost–packer interactions at every budget rather than only the smallest one. SPICE is especially close: it combines Fisher information with a penalty for gradient conflict (Chang et al., 2026). Our target-gradient proxy is not presented as new. The distinction under study is joint physical-pack construction, protected constraints, and an actual-update gate. GIST shows that diagonal optimizer geometry can be misleading for LoRA and instead scores examples in a coupled target-gradient subspace (Min et al., 2026); UDS provides a low-overhead online utility–diversity selector without reference gradients (Zou et al., 2026). Both are required comparators, and GIST scoring is also crossed with our pack-cost planner. P2D jointly selects task-sensitive attention heads and examples while charging selection and training latency (Chen et al., 2026). Because changing the trainable subspace changes both gradient geometry and cost, we treat P2D as an end-to-end comparator rather than attributing its gains to data selection alone. PartitionSel couples validation-gradient matching across domains under partition-matroid coverage constraints (Agrawal et al., 2026). Its source-constrained selector is a close matched-cost baseline; our separate question is whether profiling physical pack cost changes the feasible choice. HARP lowers train–evaluate selection cost by scoring representative leaves of a data hierarchy and inferring the rest with empirical-Bayes posteriors (Wang et al., 2026); like every selector here, its measurement runs are charged to the same accelerator-second ledger. Q-Tuning jointly prunes samples and response tokens (Wang et al., 2025b). Token pruning changes $\mathcal{R}_i$, hence both N_B and packed cost, so it is compared only inside the same frozen response-token window.

Online allocation. Dynamic bilevel weighting (Yu et al., 2025), source-mixture optimization (Tan et al., 2026), capability equilibrium adaptation (Ming et al., 2026), online per-example reweighting (Zhao et al., 2026), and bandit-based mixture scheduling (Shin et al., 2026) all recognize that data value changes during training. Layer-wise studies find family-specific gradient patterns even when relative quality trends persist (Li et al., 2026). Unlike source-level mixture adaptation, our decision variable is a complete physical pack plan for one optimizer step.

Compute-aware selection. Compute-constrained data selection charges selection and fine-tuning jointly (Yin & Rush, 2025). We adopt that accounting principle and extend the decision to a profiled pack cost. Merely adding this cost to an existing selector is not sufficient evidence of novelty: the central experiment must show an interaction between which examples are selected and how they are packed. Packing systems already optimize execution after examples are chosen. Hierarchical Balance Packing profiles multiple length and parallelism groups (Yao et al., 2025), while SlimPack balances variable-length forward and backward work with fine-grained slices (Liu et al., 2025). They are systems comparators in our study. Neither supplies evidence that a data selector should change its subset because of profiled physical cost. Threshold Filtering Packing explicitly selects related and diverse examples for a common pack (Dong et al., 2025), and packing analyses compare related versus unrelated co-packing across model and dataset scales (Wang et al., 2025a). Those methods permit or study cross-example context. Our boundary-masked setting instead holds the gradient for a fixed subset invariant and couples selection to packing only through measured feasible cost.

3. Problem Setting

Let $\theta_t \in \mathbb{R}^p$ be the trainable parameters at step t; p may denote an adapter subspace rather than all model parameters. Candidate instruction $i = (x_i, y_i)$ has packed length n_i and supervised response-token indices $\mathcal{R}_i$. Define its unnormalized supervised loss $\ell_i(\theta) = \sum_{r \in \mathcal{R}_i} -\log p_\theta(y_{ir} \mid x_i, y_{i,<r})$ and $N_B = \sum_{i \in B} |\mathcal{R}_i|$. For an accumulation window B, the implementation uses the response-token-

normalized loss

$$L_B^{\mathrm{tr}}(\theta) = \frac{1}{N_B} \sum_{i \in B} \ell_i(\theta). \tag{1}$$

The global denominator is fixed across microbatches so that, with dropout disabled, isolated examples have additive gradients.

Reference distributions are represented by source-defined dataset slices $V_0, V_1, \ldots, V_C$. V_0 measures the target behavior; $V_{1:C}$ are protected slices. Their differentiable losses are $L_c(\theta)$. V^{score} may be queried repeatedly by the scheduler, V^{gate} supplies fresh post-selection checks, and V^{audit} is intended to remain sealed until every hyperparameter and stopping rule is frozen. The repository contract requires source-, template-, and reviewed-cluster disjointness, but same-user file execution does not enforce sealing; a site-local access-isolation attestation is a real-run blocker.

A packing assignment π maps examples to physical packs and obeys

$$\sum_{i:\pi(i)=j} n_i \le L_{\max} \quad \text{for every pack } j. \tag{2}$$

Every method uses the same fixed capacity of P physical packs, P microbatches, and world size per optimizer update, and must satisfy $N_{\min} \le N_B \le N_{\max}$. Thus the scheduler cannot obtain a different Adam step count or accumulation window merely by selecting more packs. Boundary-masked attention and reset position identifiers prevent cross-example information flow. Let $\overline{C}_t^{\mathrm{all}}(B, \pi)$ denote a candidate reserve intended to upper-bound allocated accelerator-seconds for packing, reference gradients, candidate scoring, solving, gate forwards, training, and rejected updates, with sunk round costs assigned to every candidate. CPU-seconds and wall time are separate Pareto endpoints, not silently converted into accelerator cost. The goal is not to minimize example count, but to maximize target improvement subject to protected-loss and accelerator-cost constraints.

Under exact isolation, packing cannot change G_B for a fixed B. Define

$$\overline{C}_t^{\mathrm{all},\star}(B) = \min_{\pi \text{ feasible}} \overline{C}_t^{\mathrm{all}}(B, \pi). \tag{3}$$

An exact joint optimizer is therefore equivalent to selection under this non-additive profiled cost. Our joint beam search approximates that profile; it does not claim that packing order changes the gradient. The decisive ablation separates cost profiling from pack-solver quality.

4. Method

At each refresh round we sample a candidate pool, construct feasible pack plans with a beam-search heuristic, score those plans with a frozen optimizer surrogate, and select the best plan found within a fixed search budget. We then compute the selected window's exact gradient, construct the exact AdamW displacement without committing optimizer state, and apply an independent gate. Rejected work is counted against the same budget.

4.1. Training Objective

For any candidate window B, let

$$\Delta_t(B) = \theta_{t+1}(B) - \theta_t \tag{4}$$

denote the complete optimizer displacement, including momentum, preconditioning, clipping, weight decay, and learning-rate scaling. The first-order predicted improvement on reference c is

$$S_{c,t}(B) = -\nabla L_c(\theta_t)^\top \Delta_t(B). \tag{5}$$

Thus $S_c > 0$ predicts a loss decrease. For SGD, $\Delta = -\eta \nabla L_B^{\mathrm{tr}}$ and positive gradient alignment is beneficial.

Writing $G_B = N_B^{-1} \sum_{i \in B} \nabla \ell_i$, the exact pilot AdamW displacement is not additive in per-example gradients because its candidate second moment contains the globally clipped $\overline{G}_B^{\odot 2}$ defined below. For planning only, we freeze the previous denominator and use the affine surrogate

$$\widetilde{\Delta}_t(B) = d_t - \eta_t A_t G_B, \tag{6}$$

where diagonal $A_t \succeq 0$ is fixed at the pre-update state and d_t contains known momentum and weight-decay terms. We never call $\widetilde{\Delta}$ the exact AdamW update. In particular, its raw-gradient linearity omits global clipping; clipping discrepancy is measured as part of optimizer-surrogate error.

To reduce storage, a fresh Rademacher projection $R_t \in \{\pm 1/\sqrt{k}\}^{k \times p}$ stores $u_{c,t} = R_t \nabla L_c(\theta_t)$ and $q_{i,t} = R_t A_t \nabla \ell_i$. A plan aggregates

$$q_{B,t} = \frac{1}{N_B} \sum_{i \in B} q_{i,t}, \qquad v_t = R_t d_t, \tag{7}$$

and uses

$$\widehat{S}_{c,t}(B) = -u_{c,t}^\top v_t + \eta_t u_{c,t}^\top q_{B,t}. \tag{8}$$

This includes response-token weighting and the candidate-independent optimizer term. The projection reduces matching cost, not the backward passes needed to obtain individual gradients, and any claim applies only to the trainable subspace used for the sketches.

4.2. Data Construction

The candidate pool preserves source labels, license metadata, tokenized lengths, and near-duplicate clusters. Candidate sampling uses a fixed source-coverage floor so that the

scheduler cannot permanently starve a source before it has evidence to do so. Personally identifying fields, benchmark solutions, and examples without a documented redistribution or research-use basis are excluded before any selection method sees the pool. Acquisition pins the repository revision and export-byte hash and records, for every row, a stable upstream identifier plus hashes of the canonical raw row, selected content, and a versioned Unicode-normalized form. Duplicate clusters are formed before split assignment: a normalized cluster crossing registered candidate-source, reference, or benchmark roles is quarantined in full, and a retained cluster is assigned wholly to one downstream split. Normalized equality is only an exact cleaning rule. A hash-bound preflight then blocks named PII/secret patterns and retained character-5-gram near-duplicates. This deterministic lexical screen is not a semantic or privacy certificate: reviewed multilingual, paraphrase, benchmark-leakage, and privacy audits remain separate blocking checks. Above the exhaustive limit, only a reviewed exact engine whose hash-bound shards cover all cross-record field segments can lift the lexical blocker. The committed engine is unfrozen and indexes record pairs only, so it cannot clear a real multi-field export.

Reference sets measure losses rather than benchmark exact match because the gate requires differentiability. Real execution requires the score, gate, audit, and benchmark partitions to pass closed-world source, template, and reviewed-cluster disjointness checks. The scheduling sets are explicitly treated as training resources: repeatedly querying V^{score} can overfit it, which is why no final claim uses that set.

Frozen operational capability map. The pilot defines three dataset slices: general instruction, mathematics, and code. In the mathematics setting, mathematics is the target and general instruction plus code are the two protected slices; in the code setting, code is the target and general instruction plus mathematics are protected. Every slice has its own score, gate, and untouched audit partition with the same loss definition. BigCodeBench is only a secondary code diagnostic and never a differentiable gate loss. These labels are operational dataset constructs, not complete capabilities. The general slice fixes equal mass on the four admitted SmolTalk configurations; the mathematics and code slices use the registered OpenR1 default and CommitPackFT Python configurations. Within each partition, the loss is a uniform row mean of per-row response-token-mean NLL. Candidate source mass is one third per slice.

The contract requires split assignment before losses are computed. Each complete reviewed semantic-near-duplicate cluster is hashed with the source key, pinned source revision, semantic-audit hash, and sorted member record identifiers. Within source, clusters are sorted by assignment hash and apportioned $80/5/10/5$ by Hamilton largest remainder with a fixed partition tie order. These are exact cluster counts, not row proportions. An empty required source partition blocks the pilot; no boundary moves after labels or content are inspected. This gives reproducible cluster-preserving assignment, not evidence that the reviewed clusters capture every semantic overlap.

4.3. Implementation

For each proposed plan, Equation (8) yields an estimated score. We subtract separately calibrated margins for projection, reference sampling, optimizer-surrogate error, and staleness:

$$\underline{S}_{c,t}(B) = \widehat{S}_{c,t}(B) - e_{c,t}(B). \tag{9}$$

These margins are called empirical prediction margins unless their coverage conditions are verified; an ensemble standard deviation is not a confidence bound. Because maximizing a noisy score creates selection bias, the beam returns a fixed-size shortlist. An independent reference minibatch and projection, unseen during search, re-score that shortlist. We report simultaneous shortlist coverage; absent that evidence, "lower bound" is replaced by "penalized score."

The execution-cost reserve is a profiled prediction plus a margin calibrated on independent plans. We call it an upper bound only when its simultaneous coverage target is met; otherwise it is a budget predictor and the confirmatory run cannot start. Step-norm error is bounded as $\overline{q}_t = \widehat{q}_t + e_t^q$. The ideal round-level decision is

$$\begin{aligned}
\max_{B,\pi} \quad & \underline{S}_{0,t}(B) \\
\text{s.t.} \quad & \underline{S}_{0,t}(B) \geq 0, \\
& \underline{S}_{c,t}(B) \geq \tfrac{\overline{\beta}_{c,t}}{2}\overline{q}_t(B) - \tau_{c,t}, \quad c = 1{:}C, \\
& \overline{q}_t(B) \leq \rho_t^2, \qquad \overline{C}_t^{\mathrm{all}}(B,\pi) \leq B_t^{\mathrm{rem}}, \\
& N_{\min} \leq N_B \leq N_{\max}, \quad |\operatorname{range}(\pi)| \leq P, \\
& \text{Equation (2) holds,} \\
& \text{and source-coverage constraints hold.}
\end{aligned} \tag{10}$$

Here $\overline{q}_t$ estimates a trust-radius bound on $\|\widetilde{\Delta}_t\|^2$, $\tau_{c,t}$ is an allowed one-step reference-loss increase, and B_t^{rem} reserves the worst-case gate cost before selection. A no-op plan is always feasible. The binary selection, packing, norm, and non-additive cost terms make Equation (10) NP-hard. Our beam search retains the best partial plans by cost and pack-count buckets, then performs drop–swap repair. We claim no approximation ratio and compare against an exact small-pool oracle. A separate null action has $\Delta = 0$, zero score and norm, and no optimizer step or moment increment; it is not Equation (1) with $N_B = 0$. Actual allocated accelerator-seconds are metered. A reserve overrun is retained as a protocol failure; before sealed audits open, every

paired method is restarted from the common initial state under a newly frozen larger reserve. A run thus ends in exactly one way: after $T_{\max}$ rounds; when the next round's reserve exceeds the remaining budget, with the unspent remainder reported as undershoot rather than reallocated; or at a retained overrun.

Actual-update gate. After one backward pass on the selected window, form the single post-unscale, post-reduction gradient used by the frozen optimizer,

$$\overline{G}_B = G_B \min\{1, g_{\max}/\|G_B\|_2\}, \tag{11}$$

with the multiplier defined as one when $G_B = 0$ and $g_{\max} = \infty$ denoting no clipping. We construct AdamW moments $m^+ = \beta_1 m + (1-\beta_1)\overline{G}_B$ and $v^+ = \beta_2 v + (1-\beta_2)\overline{G}_B^{\odot 2}$ without committing them. Bias-corrected moments define direction $a_t = -\widehat{m}^+/(\sqrt{\widehat{v}^+} + \epsilon) - \lambda\theta_t$ and candidate displacement $\Delta_t(\alpha) = \alpha\eta_t a_t$. For the finite grid $\mathcal{A} = \{2^{-j} : j = 0, \dots, J\}$, fresh gate examples evaluate

$$D_{c,j}(\alpha) = \phi_c(z_j; \theta_t + \Delta_t(\alpha)) - \phi_c(z_j; \theta_t). \tag{12}$$

Here ϕ_c is example-average NLL clipped to $[0, M_\phi]$ solely for the gate. For each protected slice c, one fresh block is drawn with replacement from its frozen gate law before inspecting any scale. Blocks are independent across slices and rounds, while every $\alpha \in \mathcal{A}$ shares the same paired block within (c, t). With n_c examples per block and at most $T_{\max}$ rounds, define

$$U_{c,t}(\alpha) = \overline{D}_{c,t}(\alpha) + M_\phi \sqrt{\frac{2\log(C|\mathcal{A}|T_{\max}/\delta_g)}{n_c}}. \tag{13}$$

The union-bound interpretation requires fixed $T_{\max}$ and fresh gate draws; otherwise Equation (13) is only a heuristic. We commit the largest α with $U_{c,t}(\alpha) \le \tau_{c,t}$ for all protected slices, applying learning rate $\alpha\eta_t$ while committing m^+, v^+ and the step counter exactly once. If none passes, all parameter, optimizer, and scheduler state is restored and the charged update is discarded. Target improvement remains a planning criterion; it is not certified unless $c = 0$ is included in the gate. For each protected slice, a predeclared ledger enforces $\sum_{t \le T} \tau_{c,t} \le b_c$. The single global δ_g in Equation (13) already union-bounds all slices, scales, and rounds. This composition concerns only the clipped gate estimand. Final unclipped audit NLL remains an empirical endpoint, not a consequence of the gate.

4.4. Conditional One-Step Control

Proposition 4.1. *Suppose L_c is β_c-smooth on the segment from θ_t to $\theta_t + \Delta$. Assume jointly that $-\nabla L_c(\theta_t)^\top \Delta \ge \widehat{S}_c - r_c$, $\|\Delta\|^2 \le \overline{q}$, and $\beta_c \le \overline{\beta}_c$. If*

$$\widehat{S}_c - r_c \ge \tfrac{\overline{\beta}_c}{2}\overline{q} - \tau_c, \tag{14}$$

Algorithm 1 One Scheduling and Update Round

```
Sample candidates within the remaining scoring budget.
Compute lengths, trainable-subspace gradients, and sketches.
Build feasible isolated pack plans with beam search.
Re-score a fixed shortlist on independent references and select.
Compute its exact gradient, moments, and scaled AdamW candidates.
for c = 1, ..., C do
    Draw a fresh protected block Z_{c,t}; freeze it across scales.
end for
for α ∈ A from largest to smallest do
    Evaluate protected losses on all frozen round blocks.
    if all upper statistics are below their tolerances then
        Commit Δ_t(α) and optimizer state; break.
    end if
end for
Charge scoring, solving, gating, training, and rejection costs.
```

then $L_c(\theta_t + \Delta) - L_c(\theta_t) \le \tau_c$.

The proof is the descent lemma and appears in Appendix A. The proposition must use the actual displacement; substituting Equation (6) invalidates it. It controls one specified reference loss, not accuracy, harmlessness, population generalization, or multiple steps without accumulated tolerances. The practical gate controls a clipped empirical loss under its own conditions; it does not establish the score, norm, and curvature event required by Proposition 4.1.

5. Experiments

The confirmatory study plans three hypotheses. **H1**: joint selection–packing gives a better target-loss/worst-protected-loss frontier than the same score followed by best-fit packing. **H2**: any gain remains after charging selector, gate, and rejected-update costs. **H3**: the actual-update gate reduces audit regressions relative to proxy-only constraints. Failure of any hypothesis is a substantive negative result. Only the exploratory H1 mechanism factorial is frozen in the committed pilot; H2, H3, and every confirmatory acceptance threshold remain planned rather than predeclared. The primary statistic is the paired difference in dominated hypervolume over a fixed tolerance and budget grid with a fixed reference box. A future confirmatory replacement preregistration is intended to require a simultaneous 95% lower bound above zero; the committed exploratory pilot cannot declare H1 success. H3 compares protected regression at a planned noninferior target-NLL change so that shrinking all updates cannot win trivially.

5.1. Experimental Setup

We freeze a source-labeled, license-audited instruction mixture before tuning. Two target settings—mathematics and code—swap which source is targeted and which capabilities are protected. Development uses a roughly 1–2B parameter model; confirmatory runs use a roughly 7–8B model and one second model family. The planned trainable configuration is rank-64 adapters on attention and MLP projections, BF16 AdamW, response-only loss, $L_{\max} = 4096$, and isolated packing. The pilot freezes $P = 8$ packs and microbatches per update and $N_B \in [4096, 8192]$ response tokens for every method. Exact model revisions, tokenizer hashes, data hashes, optimizer coefficients, and kernel versions must be frozen in the artifact before the first confirmatory run. The committed 20-update, 131,072-reference-token configuration is explicitly a pre-compute pilot, not the B_{50M} confirmatory design. Its gate uses 512 with-replacement draws per protected slice and a frozen empirical margin, with $\sum_t \tau_{c,t} \leq 0.2$; it makes no population-coverage claim. A formal use of Equation (13) requires a separately feasible $T_{\max}$ and recomputed sample count. Its factorial uses the pinned Qwen 1.5B checkpoint and five seeds. The pinned SmolLM checkpoint is an untouched transfer diagnostic, not an additional seed or model selected after the factorial.

Let B_{50M} be the measured accelerator cost of random, isolated, best-fit-packed SFT on 50 million response tokens. We evaluate total budgets $\{0.25, 0.5, 1.0\}B_{50M}$. Each method stops at the same allocated accelerator-second budget, including accelerator preprocessing and validation. CPU-seconds and wall time are reported separately. Final common evaluation is excluded from training cost; every method-specific query is included.

Baselines are random plus identical packing, representation-similarity selection, LESS, FisherSFT, influence distillation, SPICE, DynamixSFT, a source-level dynamic allocator, GIST, UDS, PartitionSel, P2D, online target-gradient alignment followed by packing, and the proposed joint scheduler. Every feasible baseline is reported both ungated and with the identical actual-update gate. The decisive decoupled comparator uses the same candidates, scores, protected constraints, and training code, differing only by additive selection followed by deterministic best-fit packing. Candidate streams, initial checkpoints, data order seeds, and hyperparameter search budgets are paired. Table 1 shows the core methods; the appendix reports the complete baseline–gate crossing, including influence distillation and the source-level allocator. Best-fit, HBP, and SlimPack-style execution policies are crossed with a fixed selector where their public implementations support the frozen models; this comparison measures systems efficiency, not selection quality.

Primary endpoints are target audit NLL, the worst protected audit regression

$$\max_{c \geq 1} \left[L_c^{\text{audit}}(\theta_T) - L_c^{\text{audit}}(\theta_0) \right], \tag{15}$$

end-to-end allocated accelerator-seconds, CPU resource-seconds, wall time, and packing throughput. Downstream benchmarks are secondary because they are not optimized by the gate. We use five exploratory pilot seeds and retain every run rather than selecting the best seed. Confirmatory seed count, acceptance thresholds, and model crossing remain unfrozen.

5.2. Main Results

No ranking is inferred from Table 1. Before values are inserted, each cell must link to a run manifest whose per-round train ledger replays to the reported endpoint, whose audit NLLs reproduce from per-example loss records on the final ledger state, and to the aggregation script. The primary comparison is a paired frontier contrast between score-then-pack and joint scheduling, not the single best endpoint.

5.3. Ablations and Mechanistic Analysis

Planned confirmatory one-factor contrasts compare offline versus online refresh, score-then-pack versus joint planning, target-only versus protected constraints, SGD geometry versus frozen-Adam geometry, additive token cost versus measured cost, and proxy-only versus actual-update gating. A small fractional factorial on the development model checks the largest interactions. Sketch dimensions and refresh intervals are tuned only on that model. The core 2×2 contrast crosses additive versus profiled selection cost with deterministic best-fit versus beam-search packing, separating a better cost model from a better packing heuristic. All four cells use identical candidates, subset moves, beam width, expansion count, and wall-clock solver budget. The primary mechanistic statistic is the paired difference-in-differences in dominated hypervolume, supplemented by beam error against exact 24-candidate instances. A second cross uses frozen-Adam and GIST scores with the same packing paths.

Mechanistic diagnostics compare sketches with exact adapter-space dot products, surrogate and actual AdamW displacements, first-order predictions and realized gate losses, predicted and measured pack cost, solver plans and a small-pool exact oracle, and score-set versus sealed-audit regressions. If packing costs are effectively additive or joint plans match score-then-pack, the packing-aware claim is removed.

5.4. Robustness and Generalization

Robustness tests transfer the frozen scheduler configuration across model size and family, perturb source proportions,

Table 1. Planned confirmatory matched-budget endpoints. Lower target NLL, protected regression, and cost are better; higher throughput is better. Dashes are intentional: no measurements had been run when this draft was written.

Method	Target audit NLL	Worst protected ΔNLL	Accelerator-seconds	Packed tokens/s
Random + best-fit packing	–	–	–	–
Representation similarity + best-fit	–	–	–	–
LESS + best-fit packing	–	–	–	–
FisherSFT + best-fit packing	–	–	–	–
SPICE + best-fit packing	–	–	–	–
GIST + best-fit packing	–	–	–	–
UDS + best-fit packing	–	–	–	–
PartitionSel + best-fit	–	–	–	–
DynamixSFT	–	–	–	–
Online alignment + best-fit packing	–	–	–	–
Joint scheduler without gate	–	–	–	–
Joint scheduler with gate	–	–	–	–

tighten each protected tolerance, and replace reference templates with semantically matched held-out templates. Numerical controls compare isolated packed and unpacked gradients with dropout disabled. Contamination audits combine exact, normalized, and semantic near-duplicate search between candidates, all references, and final benchmarks. Results are disaggregated by source, length decile, language, and license class; no subgroup conclusion is made where sample support is inadequate.

6. Limitations

The method may spend more compute scoring and gating than it saves in training. Its pack heuristic is neither optimal nor approximation-guaranteed. Frozen Adam geometry can rank windows incorrectly, while computing individual gradients may dominate runtime. Reference losses are incomplete proxies for capabilities and can themselves be gamed; the three source slices have limited construct validity, and repeated score queries make them training data. A gate based on finite fresh samples can miss rare regressions, and formal simultaneous coverage may be too conservative to be useful. Repository manifests do not enforce OS-level audit-file isolation.

The conditional bound is a standard local smoothness consequence, not a safety theorem. It applies only to the measured trainable subspace and one actual update. It does not control downstream metrics, generation behavior, or long-horizon accumulation unless every tolerance and failure probability is accounted for. Finally, evidence from open 1–8B models may not transfer to full-parameter or frontier-scale training.

7. Conclusion

We defined a narrow test of whether data selection should be coupled to physical packing when SFT must preserve specified reference losses under a true compute budget. The proposed scheduler combines known gradient-proxy ingredients with non-additive pack planning and a gate on the actual optimizer displacement. The analysis clarifies what this construction can and cannot guarantee. Whether the interaction is practically valuable remains an empirical question; the decisive next step is the paired score-then-pack versus joint-planning experiment with all overhead charged.

Impact Statement

Compute-aware scheduling could reduce the resources needed to specialize language models and make controlled adaptation accessible to smaller research groups. The same efficiency can accelerate the creation of models optimized for harmful applications. Protected losses may create false assurance if they omit affected languages, communities, or failure modes, and optimization against fixed references can shift harm outside the measured set. Candidate data may contain private, copyrighted, or exploitative labor-derived material; selection does not cure those provenance problems. We therefore require license and privacy audits before scoring, disaggregated untouched audits, transparent reporting of gate coverage, and no use of the phrase "capability-safe." All selector and rejected-update compute is reported so environmental benefits are not overstated.

References

Agrawal, P., Chanda, P., Khatri, I., Ramakrishnan, G., Mishra, B., and Jawanpuria, P. K. Minibatch selection for language models via partition matroid constrained gradient matching. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/64010.

Chang, P., Zhang, J., Chen, B., Wang, C., Guo, C., Zhang, Y., Gao, Y., Xiang, J., Gao, Y., Sun, C., Chen, Y., and Kong, D. SPICE: Submodular penalized information–conflict

selection for efficient large language model training. In *International Conference on Learning Representations*, 2026. URL https://iclr.cc/virtual/2026/poster/10011078.

Chen, H., Zhang, Q., Li, L., Shen, Z., Ye, W., Gao, L., Wang, N., Fu, X., Shen, X., and Zhao, J. From parameters to data: A task-parameter-guided fine-tuning pipeline for efficient LLM alignment. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/61668.

Deb, R., Thekumparampil, K. K., Kalantari, K., Hiranandani, G., Sabach, S., and Kveton, B. FisherSFT: Data-efficient supervised fine-tuning of language models using information gain. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 12924–12943. PMLR, 2025. URL https://proceedings.mlr.press/v267/deb25a.html.

Dong, J., Jiang, L., Jin, W., and Cheng, L. Threshold filtering packing for supervised fine-tuning: Training related samples within packs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4422–4435. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.226. URL https://aclanthology.org/2025.naacl-long.226/.

He, Y., Wang, Z., Shen, Z., Sun, G., Dai, Y., Wu, Y., Wang, H., and Li, A. SHED: Shapley-based automated dataset refinement for instruction fine-tuning. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/b3cff9947cde08b4fa70652cb8ac9209-Abstract-Conference.html.

Ivison, H., Zhang, M., Brahman, F., Koh, P. W., and Dasigi, P. Large-scale data selection for instruction tuning, 2025. URL https://arxiv.org/abs/2503.01807.

Li, M., Li, Y., Li, Z., and Zhou, T. How instruction and reasoning data shape post-training: Data quality through the lens of layer-wise gradients. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33249–33299. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1536. URL https://aclanthology.org/2026.acl-long.1536/.

Liu, Y., Wu, G., Zhang, S., Zhang, W., Zhu, Q., Li, Z., and Wang, C. SlimPack: Fine-grained asymmetric packing for balanced and efficient variable-length LLM training, 2025. URL https://arxiv.org/abs/2509.26246.

Min, G., Huang, T., Wan, K., and Chen, C. GIST: Targeted data selection for instruction tuning via coupled optimization geometry. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/64770.

Ming, C., Qu, C., Duan, X., Pei, Q., Pan, Z., Li, Y., He, C., and Wu, L. IDEAL: Data equilibrium adaptation for multi-capability language model alignment. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/0ced4d30a9e2d50312991c6cd131a9f5-Abstract-Conference.html.

Nayak, N., Rodriguez-Diaz, P., Hulkund, N., Beery, S., and Alvarez-Melis, D. A critical look at targeted instruction selection: Disentangling what matters (and what doesn't). In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/65407.

Nikdan, M., Cohen-Addad, V., Alistarh, D., and Mirrokni, V. Efficient data selection at scale via influence distillation. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/d2d4f6858cc2d21fd0230244fcb34f1d-Abstract-Conference.html.

Pang, J., Di, N., Zhu, Z., Wei, J., Cheng, H., Qian, C., and Liu, Y. Token cleaning: Fine-grained data selection for LLM supervised fine-tuning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47837–47858. PMLR, 2025. URL https://proceedings.mlr.press/v267/pang25a.html.

Shin, H., Ji, L., Liu, X., Yu, Z., Yoo, H., Chen, Q., and Gong, Y. DynamixSFT: Dynamic mixture optimization of instruction tuning collections. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 39590–39603. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1972. URL https://aclanthology.org/2026.findings-acl.1972/.

Tan, H., Wu, S., Chen, Y., Xia, J., Xie, R., Xia, B., Sun, X., and Qi, X. Fast data mixture optimization via gradient descent. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/01d845baaa31b1043e5f280592d57dc4-Abstract-Conference.html.

Wang, N., Zhang, Z., Tang, M., Gao, Y., Cardie, C., and Galhotra, S. HARP: Efficient data selection for finetuning large language models, 2026. URL https://arxiv.org/abs/2606.07690.

Wang, S., Wang, G., Wang, Y., Li, J., Hovy, E., and Guo, C. Packing analysis: Packing is more appropriate for large models or datasets in supervised fine-tuning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 4953–4967. Association for Computational Linguistics, 2025a. doi: 10.18653/v1/2025.findings-acl.256. URL https://aclanthology.org/2025.findings-acl.256/.

Wang, S., Wang, J., Zhang, J., Wang, C., Min, Y., Wen, Z., Ren, X., Huang, F., Jiang, H., Lin, J., Liu, D., and Zhang, L. Winning the pruning gamble: A unified approach to joint sample and token pruning for efficient supervised fine-tuning, 2025b. URL https://arxiv.org/abs/2509.23873.

Xia, M., Malladi, S., Gururangan, S., Arora, S., and Chen, D. LESS: Selecting influential data for targeted instruction tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 54104–54132. PMLR, 2024. URL https://proceedings.mlr.press/v235/xia24c.html.

Yang, Y., Mishra, S., Chiang, J. N., and Mirzasoleiman, B. SmallToLarge (S2L): Scalable data selection for fine-tuning large language models by summarizing training trajectories of small models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97fe251c25b6f99a2a23b330a75b11d4-Abstract-Conference.html.

Yao, Y., Tan, J., Liang, K., Zhang, F., Hu, J., Wu, S., Niu, Y., Gong, R., Lin, D., and Xu, N. Hierarchical balance packing: Towards efficient supervised fine-tuning for long-context LLM. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://arxiv.org/abs/2503.07680.

Yin, J. O. and Rush, A. M. Compute-constrained data selection. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=4es2oO9tw1.

Yu, Y., Han, K., Zhou, H., Tang, Y., Huang, K., Wang, Y., and Tao, D. LLM data selection and utilization via dynamic bi-level optimization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 72995–73008. PMLR, 2025. URL https://proceedings.mlr.press/v267/yu25g.html.

Zhao, W., Chen, Y., Tang, Y., Ma, W., Hu, S., Hu, S. X., Iacob, A., Mehrotra, A., and Lane, N. D. Rethinking data curation in LLM training: Online reweighting offers better generalization than offline methods. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=UFwnsmFZ6R.

Zou, H., Mao, Y., Qu, Y., Wang, Q., and Ji, X. Utility-diversity aware online batch selection for LLM supervised fine-tuning. In *International Conference on Machine Learning*, 2026. URL https://icml.cc/virtual/2026/poster/64444.

A. Reproducibility Details

A.1. Proof of Proposition 4.1

By β_c-smoothness along the update segment,

$$L_c(\theta_t + \Delta) \le L_c(\theta_t) + \nabla L_c(\theta_t)^\top \Delta + \frac{\beta_c}{2}\|\Delta\|^2 \tag{16}$$

$$\le L_c(\theta_t) - (\widehat{S}_c - r_c) + \frac{\overline{\beta}_c}{2}\overline{q} \tag{17}$$

$$\le L_c(\theta_t) + \tau_c. \tag{18}$$

The first inequality is deterministic. Any probabilistic claim enters only through the joint event that the assumed score, norm, and curvature bounds hold.

A.2. Required Run Manifest

Every run records the base checkpoint and revision, tokenizer hash, trainable parameter names, optimizer and scheduler state, precision, gradient clipping, random seeds, data and deduplication hashes, candidate stream, score/gate/audit split identifiers, sketch seed and dimension, beam width, cost-model version, packing order, kernel and compiler versions, hardware identifier, accepted and rejected displacements, and cumulative accelerator time. A failed or preempted run remains in the ledger.

A.3. Gate Feasibility and Protocol Manifest

Even when the observed mean change is zero, the gate in Equation (13) requires approximately

$$n_c \ge \frac{2M_\phi^2 \log(C|\mathcal{A}|T_{\max}/\delta_g)}{\tau_{c,t}^2}. \tag{19}$$

Any formal confirmatory configuration must freeze $C, M_\phi, J, T_{\max}, \delta_g, b_c$, every $\tau_{c,t}$ and n_c, the sampling seeds, and disjoint (c,t) gate blocks, with $\sum_{t \le T_{\max}} \tau_{c,t} \le b_c$. It records the derived radius and total required examples. If $\widehat{c}_{\mathrm{fwd}}$ is the measured per-example forward cost, the round reserves

$$R_{\mathrm{gate}} = \widehat{c}_{\mathrm{fwd}}(|\mathcal{A}| + 1)\sum_{c=1}^{C} n_c. \tag{20}$$

No round starts unless scoring, candidate-update, and worst-case gate reserves all fit the remaining budget. If this is infeasible, the simultaneous-control condition is removed and any cheaper gate is labeled heuristic; H3 then remains an empirical gate comparison only. The repository protocol runner `python3 scripts/run_experiment_pipeline.py --plan experiments/sft_pipeline.json` enumerates acquisition, processing, model construction, 20-update exploratory training, and sealed pilot evaluation artifacts. The full B_{50M} confirmatory configuration is not yet frozen. It hashes the exact SFT section of `experiments/pilot_preregistration.json`, including the fixed eight-pack window, optimizer, selector, gate, cost-failure rule, and factorial contrast. Its software audit independently enumerates (B, π) pairs and, separately, minimizes a numeric nonlinear pack cost over π before selecting B; equality is therefore not implemented by calling the same subset predicate twice. The public-source registry pins full dataset and checkpoint commits; its automated license metadata check is only a preflight, and real acquisition is blocked until every manual provenance, terms, privacy, and leakage review is approved. After approval, `scripts/materialize_public_data.py` accepts only hash-pinned local JSONL exports, emits row-level provenance and retained role-specific files, and verifies their artifact hashes. Its committed synthetic fixture contains within-role and cross-role duplicates and succeeds only when the former is collapsed and the latter is quarantined; it downloads no public rows and is not experimental evidence. `scripts/run_content_gates.py` additionally binds `experiments/content_gates.json` to those artifacts and fails closed on retained named PII/secret patterns or lexical near-duplicates. Its exhaustive small-artifact path can pass; the non-recall-complete MinHash-LSH path for larger artifacts is inconclusive by itself. `scripts/validate_external_overlap_audit.py` now defines the only large-artifact completion route: records are ordered by identifier, unordered pairs receive row-major upper-triangle ordinals, and contiguous shard ranges must cover exactly $n(n-1)/2$ comparisons without gaps or overlaps. Each shard binds the materialization, gate specification, reviewed engine source, runtime image, and review record; approximation,

early termination, shard errors, or any qualifying pair blocks the ledger. Reported pair identities, roles, thresholds, and scores are independently checked without persisting matched text. A temporary disk-backed exact join also enumerates every pair sharing a 5-gram and rejects omitted findings; this is recall-complete for the positive frozen Jaccard thresholds because any qualifying pair must share a gram. Shard coverage is therefore an engine-governance attestation, whereas readiness depends on reproducing the complete qualifying-pair set. The committed engine hashes remain null blockers, so this contract is executable only as a generated software self-test and no large real artifact currently passes. Neither lexical route is evidence of semantic independence or completed manual review. Moreover, this external contract indexes record pairs, not canonical field segments. It cannot clear a multi-field export even after its engine fields are filled; a reviewed successor must cover the complete cross-record segment-pair space. `experiments/overlap_projection_contract.json` now enumerates every pilot source/role and requires separate model-input, training-target, protected-input, and protected-target slots as applicable. Its validator binds each inventory to source revision, configuration, split, exporter, canonical field paths, and a review-artifact hash. Runtime enumeration requires every textual leaf path to be projected or explicitly excluded; generated attestations cannot be relabeled as real. The generated two-field fixture is only a schema audit. All eight real field inventories and review records remain null, so concatenating one convenient text field cannot clear the overlap gate. The separate `experiments/sft_reference_contract.json` fixes the capability map and domain-separated cluster-hash split. The process-stage validator loads the hash-bound semantic-audit artifact, requires every retained candidate record exactly once, checks source/configuration/role provenance, reproduces hash-sorted cluster ranks and largest-remainder assignments, and requires exact equality with partition JSONL. Its generated self-test rejects a wrong partition, duplicate cluster, stale contract, omitted retained cluster, and generated identifiers relabeled as real; 19/20/21-cluster boundary allocations are also fixed. The self-test contains identifiers only and is not empirical evidence. The shared acquisition-to-retention validator now recomputes row identifiers, raw and projected-content hashes, canonical duplicate representatives, and every retained/suppressed/quarantined disposition from acquisition bytes. `scripts/validate_sft_train_ledger.py` now closes every factorial endpoint row against its run manifest and hash-bound per-round train ledger, and `aggregate_sft_factorial.py` refuses any row that does not close. The replay checks a contiguous parameter-state and optimizer-step chain from the common initial state, with one step per accepted update and none for a rejected update or null action. It also checks reserve admission before every round, charging of all seven cost components (packing, reference gradients, scoring, solving, gate forwards, training, and rejected updates), and the three-way stop rule above. Gate evidence must use a fresh block per protected slice and round with the frozen 512 draws. The upper statistic is recomputed as the observed mean plus the frozen empirical margin, and the committed scale must be the largest passing one. The cumulative tolerance ledger is also checked. Finally, the audit endpoint must be evaluated on the ledger's final state, from which the row's target and worst-protected NLL changes are recomputed. Each declared audit NLL is itself recomputed from hash-bound per-example records carrying the example identifier, slice, evaluated state hash, response-token count, and summed NLL. The validator applies the frozen estimand, a row-uniform mean of per-example response-token-mean NLL, and requires initial and final records to be evaluated on the exact initial and final states and to cover exactly the audit partition named by the hash of its sorted identifiers. A protocol-failure run must leave its sealed audits unopened. Paired rows must share one split-manifest hash and audit partition per slice, the reference cost within a target setting, and the initial state and initial audit NLLs within seed and target setting. Twenty-two generated mutations fail, including a hidden charge, a rejected update that advances the optimizer, a smaller-than-largest passing scale, a reused gate block, a dropped round, an unmarked overrun, a final audit record evaluated on the initial state, a dropped or duplicated audit example, a failed run that opens its audit, and a token-weighted rather than row-uniform mean. The real runner now re-executes the aggregator over the hash-bound endpoint file, requires every run manifest to bind the processed split manifest, and demands exact equality with the evaluation summary. The fixtures are non-empirical, and the per-example forward passes remain site-local measurements. Real execution remains blocked on reviewed source-specific multi-field inventories and a site-local sealed-resource access attestation. Declarative stage checks do not count as execution evidence. With `--execute --synthetic-audit --commands experiments/synthetic_commands.json`, a CPU-only audit exactly enumerates packings for two length-10 packs under a quadratic load plus launch cost, checks direct joint selection against an independently computed profiled-cost form, and exercises the finite gate grid over 512 seeded score perturbations. A separate frozen Hoeffding fixture uses $M_\phi = 1$, $C = 2$, $|\mathcal{A}| = 3$, $T_{\max} = 10$, $\delta_g = 0.05$, and $\tau = 0.2$, which fixes $n_c = 355$. Every output is marked non-empirical. These fixtures test implementation invariants and a constructed non-additive-cost case; they are not evidence that the method improves SFT. `scripts/aggregate_sft_factorial.py` separately implements the frozen reference-box clipping, failed-run assignment, dominated hypervolume, cost-by-packer difference-in-differences, and paired max-t seed bootstrap. Its self-test uses only generated endpoints and cannot populate Table 1.

A.4. Numerical Packing Check

Before training, randomly sampled windows are evaluated both unpacked and with isolated packing. We compare response-token counts, losses, and every trainable-parameter gradient. A discrepancy above a predeclared numerical tolerance blocks the experiment rather than being absorbed into scheduler noise.

B. Additional Results

Table 2. Required diagnostic report; values remain pending execution.

Diagnostic	Value
Sketch versus exact inner-product error	–
Surrogate versus actual update cosine	–
Gate rejection rate	–
Empirical protected-bound coverage	–
Cost-model absolute percentage error	–
Solver overhead / total cost	–
Score–audit generalization gap	–

Secondary per-task tables, full tolerance frontiers, every seed, negative results, and examples of rejected plans will be added without filtering once the protocol is run. Exploratory analyses will be labeled and will not replace the confirmatory endpoints in Table 1.